Dockerfile Generation & Testing Manual

A Complete Engineer's Guide: Constructing, Building, and Verifying the Universal OCR Container

Document Version: 1.0.0
Author: DevOps & Systems Engineering Team
Date: 2026-10-21
Status: Production Ready

---

1. What is a Dockerfile and Why is it Generated?

A **Dockerfile** is an automated build manifest containing step-by-step instructions that assemble a self-contained, reproducible virtual environment (called a Docker Container).

In artificial intelligence and computer vision applications, Python applications rely heavily on low-level operating system C++ binaries, graphics rendering drivers (such as `libgl1` and `libglib2.0-0`), and tightly coupled numerical processing libraries (`numpy`, `torch`, `easyocr`, `opencv-python-headless`). Without containerization, moving code between different machines frequently results in:

- Missing shared object errors (`ImportError: libGL.so.1: cannot open shared object file`)
- Version conflicts between host and application Python environments
- Incompatible CUDA/CPU PyTorch builds
- Differing font rendering or path conventions between Windows and Linux

By generating a Dockerfile, the entire application stack—including the minimal Linux operating system, OpenCV graphics drivers, Python 3.10 interpreter, PyTorch CPU runtime, EasyOCR models, and Streamlit web interface—is sealed into a universal container that runs identically on any workstation, server, cloud VM, or offline enterprise datacenter.

---

2. Line-by-Line Anatomy of the Generated Dockerfile

The project `Dockerfile` is structured into six deliberate, production-hardened stages:

Stage 1: Base Operating System

```
FROM python:3.10-slim
```

- **Explanation:** Instead of using a bloated 5–10 GB full Linux distribution, this pulls an official lightweight (~150 MB) Debian Linux base image with Python 3.10 pre-installed. It minimizes memory consumption, shortens download times, and eliminates unnecessary background services.

Stage 2: Low-Level System Dependencies

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 \
    libglib2.0-0 \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

- **Explanation:** OpenCV requires native C++ shared libraries (`libgl1` and `libglib2.0-0`) to decode image matrices and execute computer vision algorithms. Installing these via `apt-get` guarantees OpenCV will never crash with missing shared library errors. The final clean-up command purges the package cache to keep image layers lean.

Stage 3: Working Directory Definition

```
WORKDIR /app
```

- **Explanation:** Establishes `/app` as the primary root folder for code execution, dependency installation, and artifact generation inside the container.

Stage 4: Dependency Caching & Installation

```
COPY requirements.txt .
RUN pip install --no-cache-dir --extra-index-url
https://download.pytorch.org/whl/cpu -r requirements.txt
```

- **Explanation:** Copies only `requirements.txt` first and executes `pip install`. This leverages Docker's layer caching mechanism: unless `requirements.txt` changes, Docker reuses the pre-built layer, making subsequent container builds virtually instantaneous. Specifying `--extra-index-url https://download.pytorch.org/whl/cpu` prevents downloading huge GPU PyTorch binaries (2.5+ GB), keeping the container lightweight (~600 MB total).

Stage 5: Application & Model Ingestion

```
COPY models/ ./models/
COPY src/ ./src/
COPY tests/ ./tests/
```

- **Explanation:** Ingests the model manifest metadata, core Python business logic (`app.py`, `ocr_engine.py`, `utils.py`), and the automated QA test suite into the container sandbox.

Stage 6: Port Exposure & Launch Entrypoint

```
EXPOSE 8501
ENTRYPOINT ["streamlit", "run", "src/app.py", "--server.port=8501", "--
server.address=0.0.0.0"]
```

- **Explanation:** Informs Docker that network traffic will flow through port 8501. The entrypoint instructs Streamlit to listen on `0.0.0.0` rather than `localhost`, allowing incoming HTTP requests from the host machine to reach the dashboard.

---

3. The Companion Configuration: `docker-compose.yml`

To eliminate the need to remember long, error-prone terminal commands with multiple flags, `docker-compose.yml` encapsulates all deployment parameters:

```yaml
version: '3.8'

services:
  ocr_app:
    build: .
    container_name: ocr_app
    ports:
      - "8501:8501"
    restart: unless-stopped
    environment:
      - PYTHONUNBUFFERED=1
```

- **Port Mapping (`8501:8501`):** Routes traffic from the host computer's port 8501 directly into container port 8501.
- **Container Naming (`ocr_app`):** Assigns a clean, deterministic name for status checks and logs.
- **Restart Policy (`unless-stopped`):** Ensures automatic recovery if the host server reboots.

With this file in place, building, launching, and managing the entire application is executed with a single command:
`docker-compose up -d --build`

---

4. Step-by-Step Protocol to Test the Docker Container

Follow these sequential steps to build, inspect, and verify the container on your system:

Step 4.1: Launch Docker Desktop

- Open **Docker Desktop** from the Windows Start menu.
- Verify the bottom-left status indicator turns **GREEN** with the label **"Engine running"**.

Step 4.2: Execute the Build & Run Command

- Open Command Prompt (`cmd`) or PowerShell.
- Navigate to the project root:

  ```
  cd C:\Pankaja\AUTOMATED_SDLC_AGENT_PROJECTS\ocr_detection
  ```

- Run the compose command:

  ```
  docker-compose up -d --build
  ```

- Docker builds the layers, installs dependencies, starts the container, and confirms: `Container ocr_app Started`.

Step 4.3: Terminal Status Verification

- Execute:

  ```
  docker ps
  ```

- Verify the output lists:
  - **IMAGE:** `ocr_detection-ocr_app` (or similar)
  - **STATUS:** `Up X seconds`
  - **PORTS:** `0.0.0.0:8501->8501/tcp`
  - **NAMES:** `ocr_app`

Step 4.4: Web Browser Functional Verification

- Open your web browser (Google Chrome, Microsoft Edge, or Firefox).
- Navigate to:
  `http://localhost:8501`
- The Streamlit OCR application will appear. Perform the following verification checks:
  1. **Benchmark Motion Blur Test:** Upload `tests/test_images/1_motion_blur.jpg` → verify text is detected and bounding boxes render correctly.
  2. **Skew Angle Test:** Upload `tests/test_images/3_skew_15deg.jpg` → confirm rotated characters are successfully parsed.
  3. **High Glare Test:** Upload `tests/test_images/5_glare_flash.jpg` → verify the engine extracts characters despite harsh illumination.
  4. **Export Verification:** Click **"Download Annotated Image"** and **"Download Results as CSV"** to ensure file generation works inside the container.

Step 4.5: Internal Automated Test Execution Inside Docker

- Execute the complete 26-test automated PyTest suite directly inside the running Linux container:

  ```
  docker exec -it ocr_app pytest tests/
  ```

- Confirm that **all 26 tests pass** natively inside the containerized Linux environment:

  ```
  tests/test_utils.py ................. [ 65%]
  tests/test_ocr_engine.py .........    [100%]
  ====================== 26 passed in 3.42s ======================
  ```

---

5. Container Lifecycle Management Commands

Use this quick-reference table for routine maintenance and operational tasks:

Operation	Command	Description
Build & Start (Detached)	`docker-compose up -d --build`	Builds image and launches container in background
Start Existing Container	`docker-compose up -d`	Starts existing container without rebuilding
Stop Container Gracefully	`docker-compose down`	Stops and removes container network cleanly
View Live Container Logs	`docker logs -f ocr_app`	Streams real-time Python stdout and Streamlit logs
Execute Test Suite Inside Container	`docker exec -it ocr_app pytest tests/`	Runs all 26 automated unit & integration tests
Open Interactive Shell Inside Container	`docker exec -it ocr_app /bin/bash`	Opens bash terminal inside Linux container
Inspect Resource Consumption	`docker stats ocr_app`	Displays live CPU %, memory usage, and network I/O
Clean Up Unused Docker Artifacts	`docker system prune -f`	Frees disk space by removing unused dangling layers